OSPREYFLOW

Developer Guide

OspreyFlow is an open-source intelligence mapping platform for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.

Version	1.10
Date	2026-11-02
Describes	OspreyFlow v0.4.1
Status	Issued

Document control

Version 1.10 · 2026-11-02 · For OspreyFlow v0.4.1 · Issued

Version	Date	What changed
1.10	2026-11-02	New §7.6, Connection strength: the stored weight's formula, the influencer schema and its resolution order, influencers applied at read time under the reader's access, the rarity query, and shipping influencers in the weights file. §9.1: the strength and rarity routes and the influencers' routes.
1.9	2026-11-01	§9.1: the API's OpenAPI document, `GET /v1/openapi.json`, signed in, its version the release.
1.8	2026-11-01	§3.8a: the Checks on load step — attaching the case's and the deployment's rules to a pipeline and choosing what a hit does there. §3.8: Verify shows the alerts its sample would raise, beside the rejections, and writes nothing. §2: a data engineer attaches rules but does not write them.
1.7	2026-11-01	The opening says what OspreyFlow is for: intelligence mapping, for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.
1.6	2026-10-29	Describes v0.4.1. No change to its content.
1.5	2026-10-28	§6.1: row numbers, counted from 1 in the Glass and from 0 in the API.
1.4	2026-10-27	§7.5: the tests behind it described, not named by path (the build's folder is not part of the product).
1.3	2026-10-27	§7.5: sources withheld for a reader whose case does not read all of an element's datasets, and what a contributor's writer must do about it.
1.2	2026-10-18	§3.3: the NIEM pack's incidents and custody links.
1.1	2026-10-15	A cover page, contents, a list of figures, and this version history.
1.0	2026-10-14	First issue, with screenshots.

Contents

1. How OspreyFlow is built, for a data engineer 5
2. The Data Engineer role 5
3. The Ingest Studio 6
   - 3.1 Pipelines 7
   - 3.2 The canvas 8
   - 3.3 Domain model 9
   - 3.4 Source and connectors 10
   - 3.5 Profile 11
   - 3.6 Map 12
   - 3.7 Load 14
   - 3.8 Verify 15
   - 3.8a Checks on load 17
   - 3.9 Publish, versions and running 17
4. Files 18
5. The ingest folder 18
6. After the load 19
   - 6.1 Held rows 19
   - 6.2 Data feeds 20
   - 6.3 Schedules and watched folders 20
   - 6.4 Reverting a load 20
7. The domain model 21
   - 7.1 Types 21
   - 7.2 What every record carries 21
   - 7.3 Inferred and confirmed 21
   - 7.4 Resolution: never merged 21
   - 7.5 Sources withheld for a scoped reader 21
   - 7.6 Connection strength: weights, influencers and rarity 23
8. Screening lists 24
9. Interfaces 25
   - 9.1 The API 25
   - 9.2 Files on the server 26
   - 9.3 Commands 26

List of figures

Figure 1 — The Ingest Studio's Pipelines list: name a pipeline, or open one 7
Figure 2 — A pipeline on the canvas: domain model, source, profile, map and load 8
Figure 3 — Choosing the domain pack: the core model, or a pack over it 9
Figure 4 — The Source stage: where the data comes from 10
Figure 5 — The profile: each column's type, gaps, distinct values, samples and pattern matches 11
Figure 6 — The map: source fields on the left, one row's entities and links on the canvas 12
Figure 7 — An entity's settings: its type, what it is matched on, and where each property comes from 13
Figure 8 — The Load stage 14
Figure 9 — Verify: what the recipe would create and reference, and the graph it would make 15
Figure 10 — Verify: the alerts the sample would raise under the pipeline's checks, beside the rejections 16
Figure 11 — Checks on load: the rules that judge this pipeline's loads, and what a hit does here 17
Figure 12 — The Studio's Files tab: images, videos and text files added to the case 18
Figure 13 — Alerts → Held rows 19
Figure 14 — Reports → Data feeds: the pipelines that load into this case, and when each last ran 20

OspreyFlow is an open-source intelligence mapping platform for crime analysis, anti-money-laundering, fraud detection, and event monitoring and alerting.

For data engineers: the people who bring data into OspreyFlow. It covers the Ingest Studio from a first pipeline to a scheduled one, the domain model data is mapped into, the rules a load must pass, and the interfaces a data engineer uses beyond the screens. It describes what is built, and says plainly where something is not yet.

The other guides in this folder are the administration guide (administration-guide.pdf), for whoever installs and runs OspreyFlow, and the user guide (user-guide.pdf), for analysts.

---

1. How OspreyFlow is built, for a data engineer

Three tiers; everything that passes between them crosses one of four typed ports:

Tier	What it is
The Glass	The browser workspace. The Ingest Studio is one of its places, shown to anyone holding `edit_pipelines`
The Conduit	The API. It holds the rights, the case levels and the audit, runs the loads, and judges each load with the case's rules
The Vault	The stores: the graph store (ArcadeDB), the analytics database (DuckDB, or ClickHouse), graph compute, the models, Valkey, the object store, OpenSearch, and PostgreSQL for the Conduit's own records

Port	What passes through it
GraphPort	Writing and finding nodes and links, reverting a load's elements, typed traversal
AnalyticsPort	Event aggregates — never raw SQL
ComputePort	Centrality, communities, ownership chains
ModelPort	Extraction, transcription, image description, embedding

Two narrower ports sit beside them: the media store, for content files, and the recognition port.

Access control lives in the stores' queries. Every read is made as a person: their clearance, handling codes and the case's datasets are part of the query itself, never applied afterwards. The Glass never sees a store's own query language or a source system's identifiers.

Three separate questions decide what someone can do with data: their **clearance and handling codes** (what they may see), their **level on a case** — Viewer, Editor or Author (where they may act), and their **rights** (what they may do). A right never widens what a person may see.

2. The Data Engineer role

The role holds	What it lets you do
`edit_pipelines`	Open the Ingest Studio; make and edit pipelines
`verify_pipelines`, `publish_pipelines`, `run_pipelines`, `schedule_pipelines`	Verify, publish, run and schedule them

The role holds	What it lets you do
`add_ingest_files`	List and upload source files
`read_held_rows`, `decide_held_rows`	See, release and reject rows a rule held
`read_data_feeds`, `operate_schedules`	See what feeds a case; Admin → Schedules and runs
`load_screening_lists`, `deploy_rules`, `read_rule_audit`, `read_alert_queue`	Screening lists and the rules that judge loads
`create_case`, `share_case`, `investigate`, `run_analysis`, `read_reports`	Work in cases alongside analysts

It also carries `revert_loads`, for reverting a load — which no screen or command uses yet (section 6.4). It attaches rules to a pipeline (section 3.8a) but does not write them: a rule written as a sentence needs the analyst's `keep_watches` as well as `deploy_rules`, so an analyst writes it in Monitoring. It does **not** hold `edit_graph`, `confirm_inferred`, `extract_files`, `resolve_entities`, `export` or `read_report_feed`: those are the Analyst's and Reports Viewer's. An administrator can grant any single right on its own.

3. The Ingest Studio

The Studio is an editor for a **recipe**: a declarative YAML description of how one source becomes part of a case's graph. It works only inside a case — with no case open it says so — and what it loads goes into that case's dataset. A recipe belongs to the first dataset of the case it was made in, and is listed, verified, published and run only in cases that read that dataset.

3.1 Pipelines

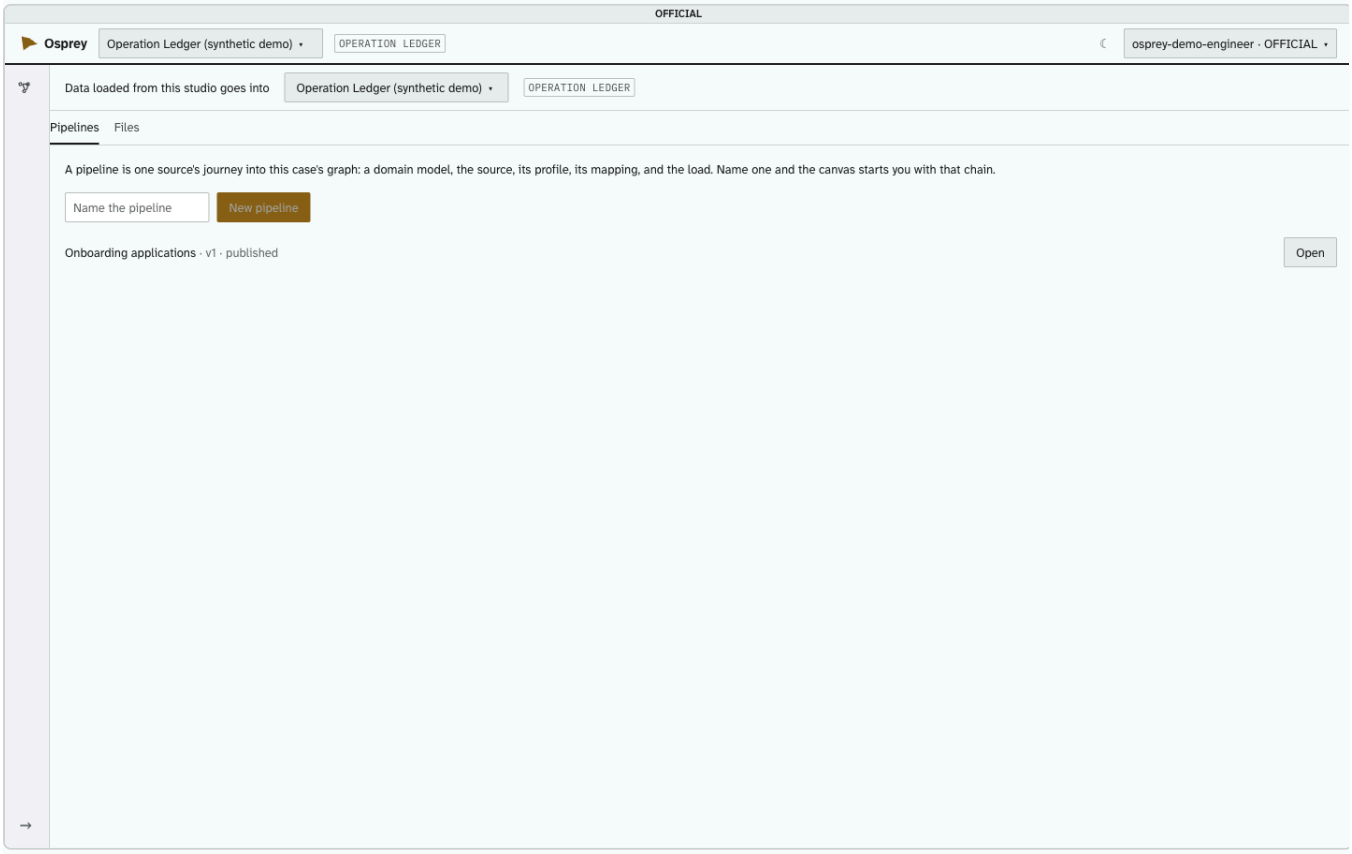


Figure 1. The Ingest Studio's Pipelines list: name a pipeline, or open one

Name the pipeline and choose **New pipeline**: the canvas opens with a whole chain to fill in. Each existing pipeline is listed with its version and status (draft, published or flagged), and **Open** takes you to it.

3.2 The canvas

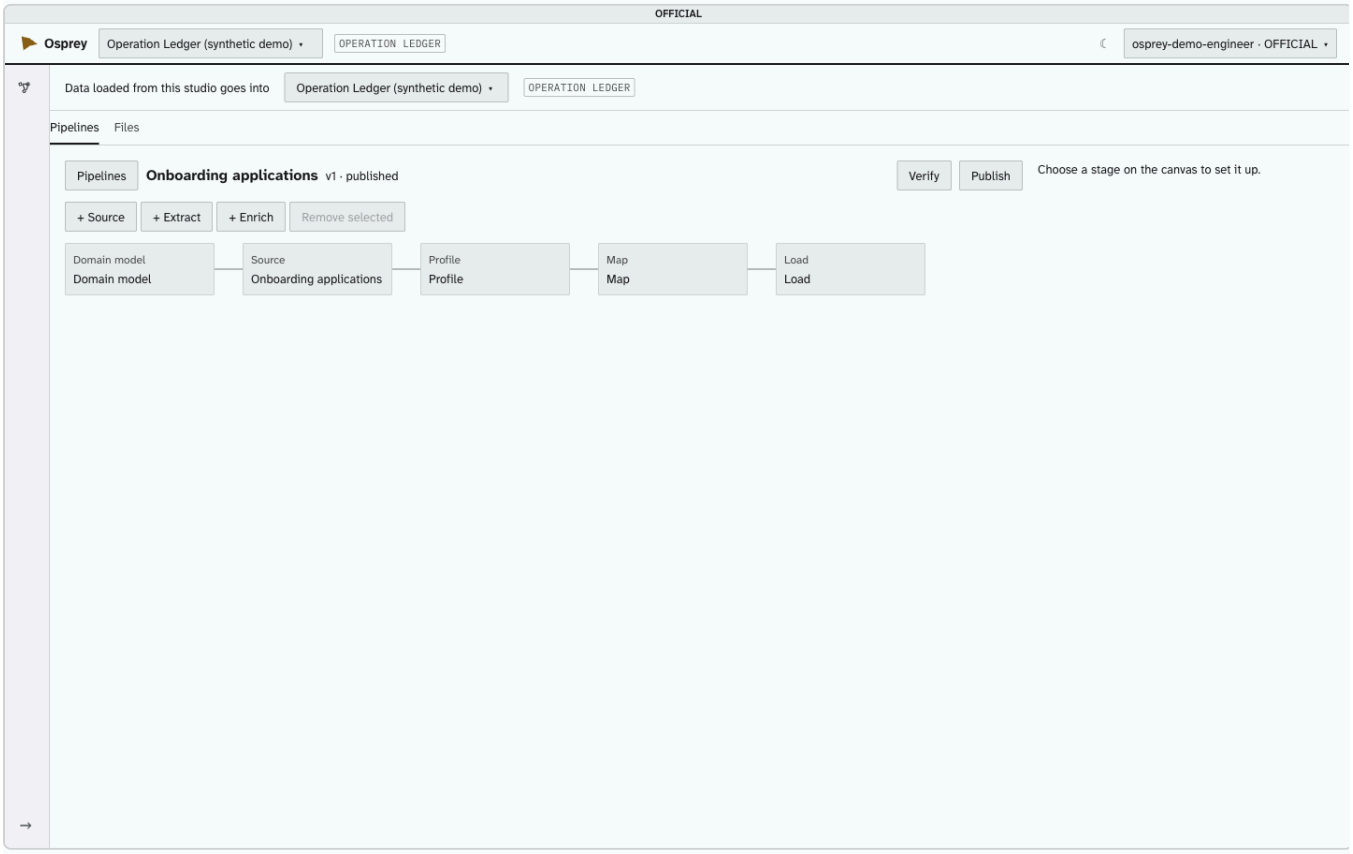


Figure 2. A pipeline on the canvas: domain model, source, profile, map and load

A pipeline is a chain of stages, each with one job:

Stage	What it does
Domain model	What the data means: the core model, and the pack it adds. Exactly one, at the root
Source	The file, its connector, its handling marking and its legal basis
Profile	What the source holds, column by column
Map	How one row becomes entities and links
Extract	Optional: finds entities in the text of Documents a map made
Enrich	Optional: geocodes places — the local gazetteer first
Load	Writes into the case's dataset. It has no settings

- **+ Source** adds another whole source → profile → map → load branch. Each source is its own load, revertible on its own, and inherits the first source's marking and legal basis.
- **+ Extract** and **+ Enrich** go after the last map. **Remove selected** never removes the domain model.
- **Verify** and **Publish** are buttons at the top, not stages.
- Every change is saved as you make it. A pipeline with a cycle, a second domain model, a source that reaches no load, or an extract not fed by a map is refused, in words, before it is saved.

3.3 Domain model

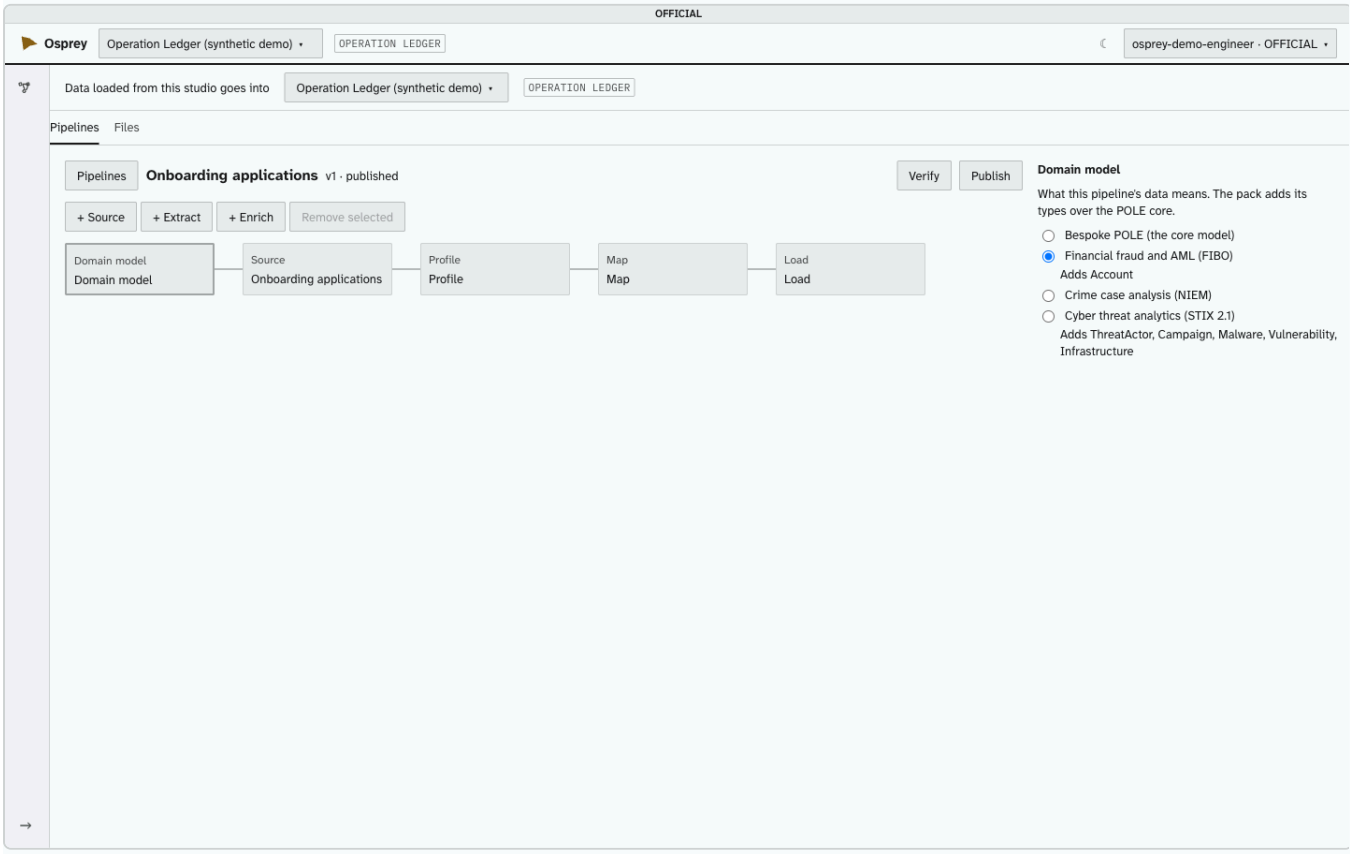


Figure 3. Choosing the domain pack: the core model, or a pack over it

Every pipeline maps into the **POLE core** — people, objects, locations, events, organisations — and may add one pack over it:

Pack	Adds
Bespoke POLE	The core only
Financial fraud and AML (FIBO)	Account; registered at, beneficial owner of (with a percentage and nature), controls, holds account
Crime case analysis (NIEM)	Incident (name, category: assault, disorder, find or other; item; when); used as, role in — each with a required role; shared a cell with (cell, dates), co-defendant of (case reference), in conflict with (reason, dates), visited (in person or video, dates)
Cyber threat analytics (STIX 2.1)	ThreatActor, Campaign, Malware, Vulnerability, Infrastructure; attributed to, targets, indicates, compromises

A pack only adds types, or widens which types a link may join; it cannot change the core.

3.4 Source and connectors

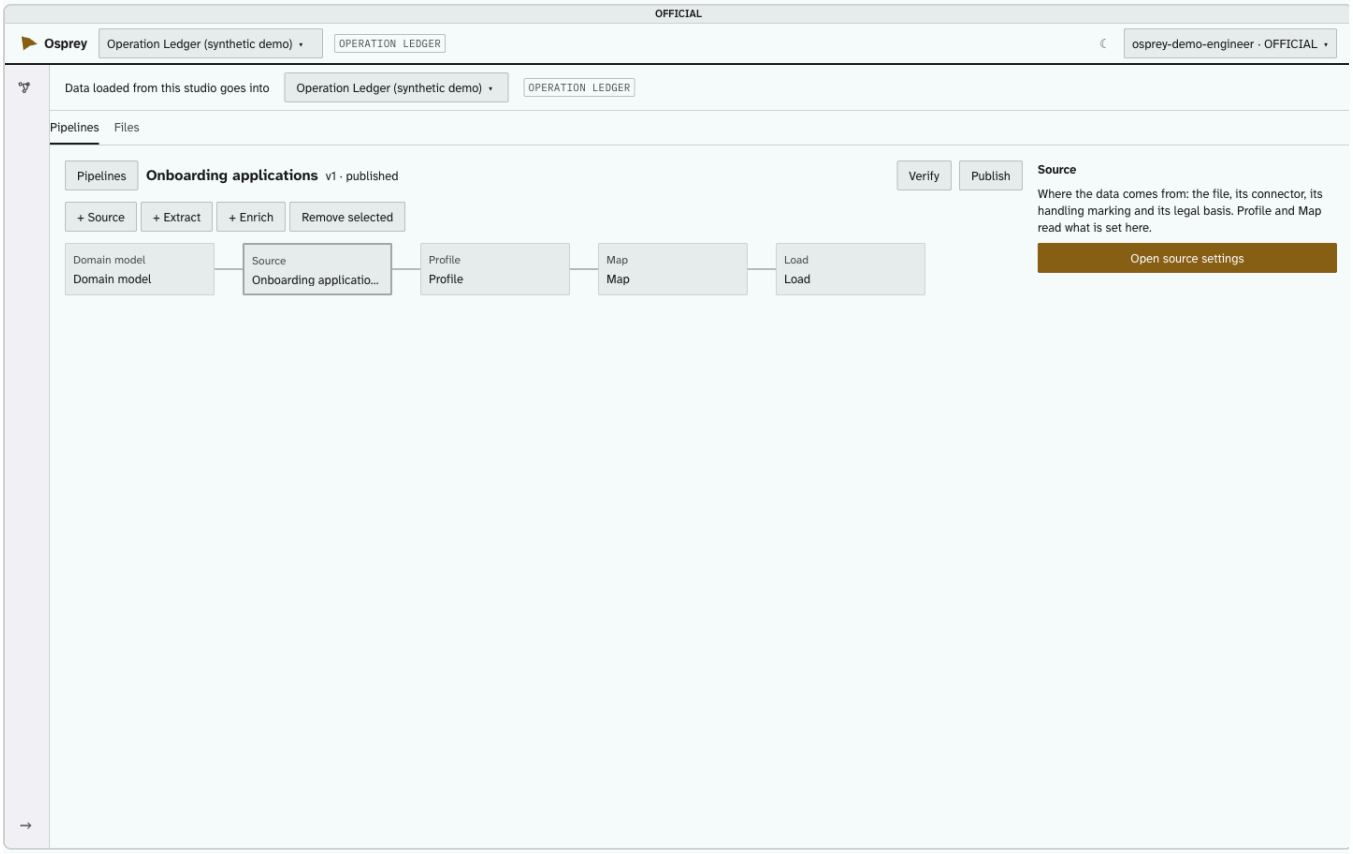


Figure 4. The Source stage: where the data comes from

Open source settings asks three things before anything else can happen:

1. **Where the data is**: a file from the ingest folder, a file from your computer (uploaded into the folder for you), one already on the server, or a path or address you type.
2. **Its handling marking**: the classification and handling code everything from it will carry. A classification left blank is recorded as the scheme's lowest level, and the screen says so.
3. **Its legal basis**: the power or consent the data is held under.

Connectors. The catalogue lists every kind of source OspreyFlow is designed for — communications records, mobile-device extractions, financial records, vehicle and movement data, generic files, documents and media — and marks each as available or not. **Available today: four connectors, all reading CSV**: generic CSV, call records (day-first dates), cell sites (WGS 84 co-ordinates) and seizure records. The others are shown as not yet available. A connector is a small profile file in `connectors/`; adding one needs the Conduit restarted.

3.5 Profile

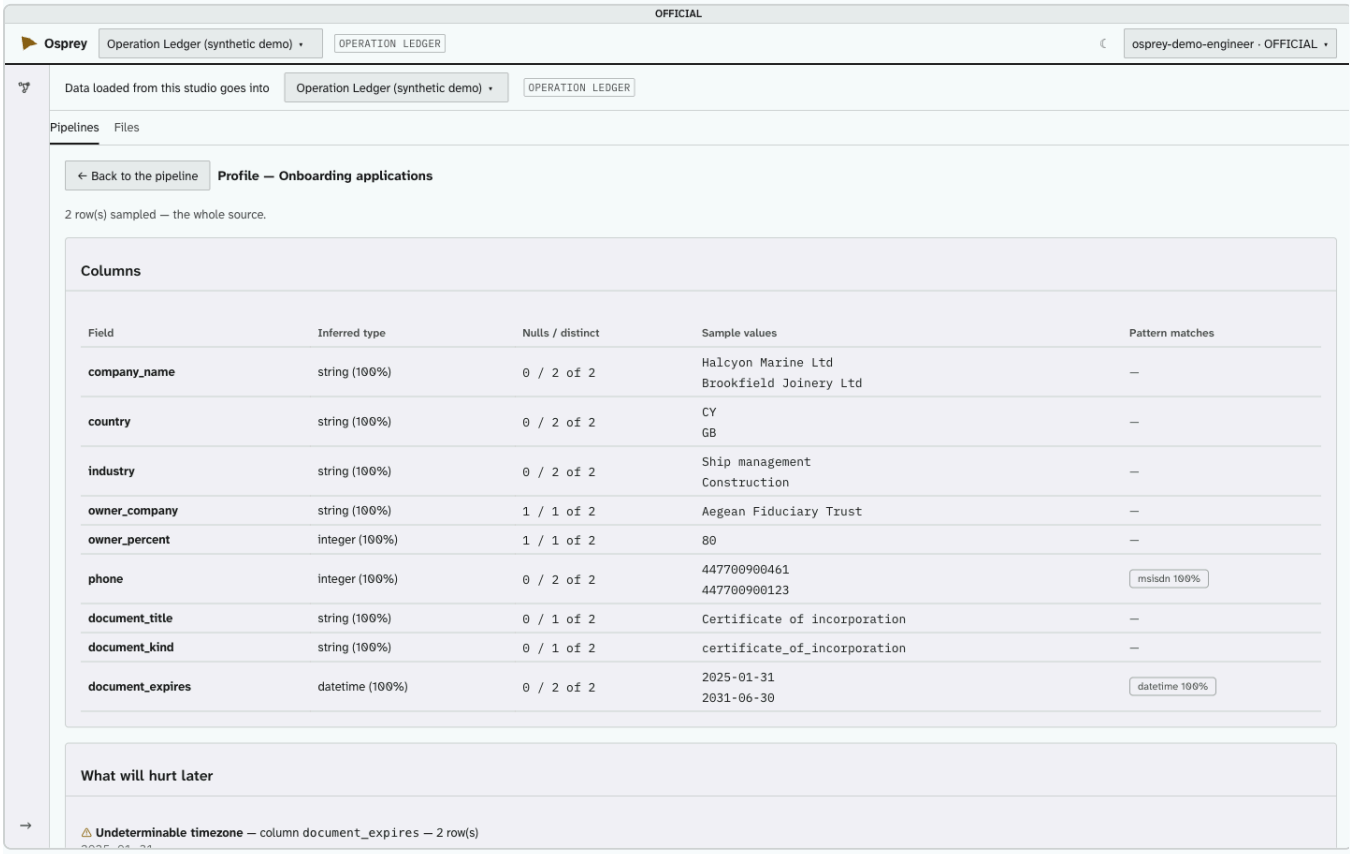


Figure 5. The profile: each column's type, gaps, distinct values, samples and pattern matches

The profile reads up to 10,000 rows (or the whole source, if smaller) and shows, for each column, its inferred type and how sure that is, how often it is empty, how many distinct values it has, five sample values, and which known patterns it matches (a phone number, an IMEI, a date…) and how often. A share is never rounded up to 100% when it is not.

What will hurt later names the problems before they reach the graph: mixed date formats, a timezone that cannot be told, an encoding other than UTF-8, rows with differing column counts, and apparent duplicates.

Someone not cleared for the source's marking sees the samples' shape only: letters as `A`, digits as `#`. Map cannot be opened until the profile has run.

3.6 Map

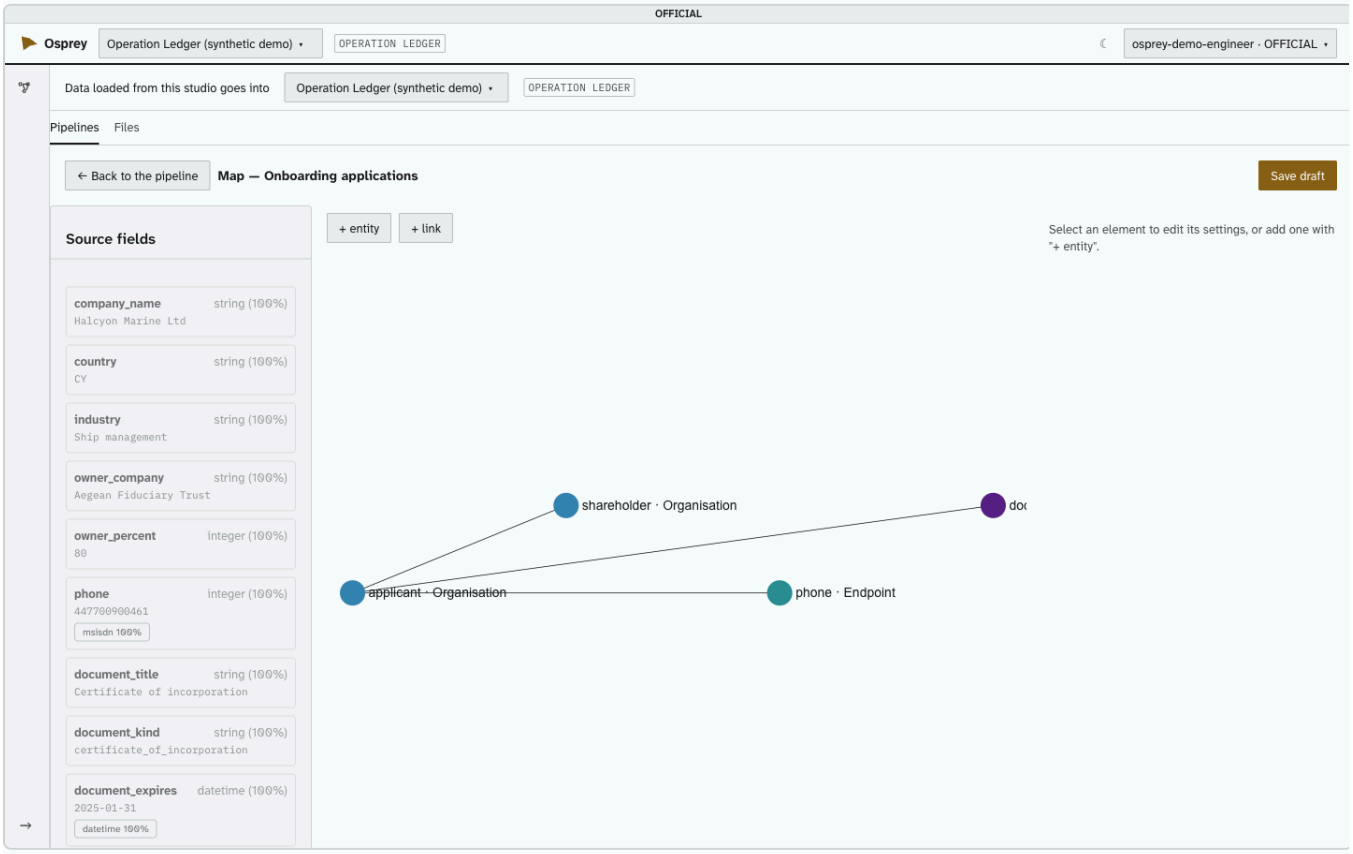


Figure 6. The map: source fields on the left, one row's entities and links on the canvas

One row of the source becomes a small graph. **+ entity** adds an entity — first its POLE class, then its type; **+ link** joins two. The source's fields are listed on the left with a sample of each; every field not yet used is shown until it is mapped or marked **ignore**.

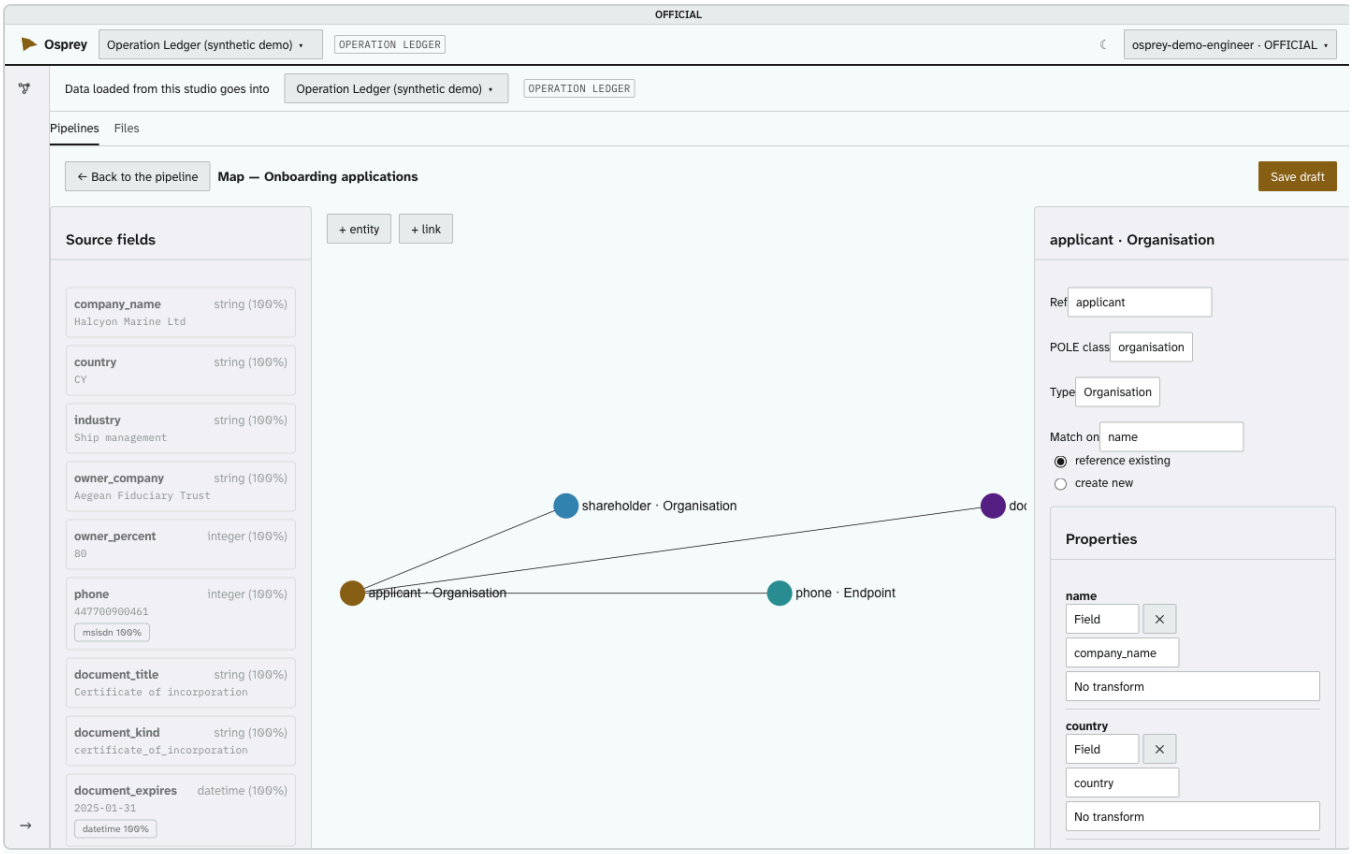


Figure 7. An entity's settings: its type, what it is matched on, and where each property comes from

For each entity:

- **Match on** — the property that identifies it — and whether a row **references an existing** entity with that value or **creates a new** one.
- **Properties**: each takes a field, several fields, a fixed value, or a value map, optionally through a transform.
- **When**: an element can depend on a field's value.
- **Content this holds**: a Document's text, an image or a video.

For each link: its type, and its time — not recorded, or taken from a column. A link with a time can mark text a model may later read.

Transforms are Knowledge Adapters, never expressions. OspreyFlow has no expression or scripting language. A transform is one of a fixed set of tested adapters: `msisdn`, `imei`, `imsi`, `email`, `datetime`, `person_name`, `address`, `iban`, `card`, `coordinate_lon`, `coordinate_lat`, `geometry_ref`, `polygon_ref`, `grid_ref` and `postcode`. Some take more than one column (co-ordinates two, a grid reference three). A structured result is bound one part at a time, such as `person_name(part=family)`. A transform that does not exist yet is a new adapter, written with golden test cases — not something typed into a recipe.

Handling and provenance default from the source; an element may **tighten** its handling, never loosen it. Data a recipe loads is *asserted* — it came from a source, not a model.

Refused at save, in words: a link of a type the ontology marks as probabilistic, `SAME_AS` or `NOT_SAME_AS` (resolution writes those, never a recipe), an unknown type, a property the ontology does not have, and a link's model-readable text with no time.

Save draft saves the map; it does not verify it.

3.7 Load

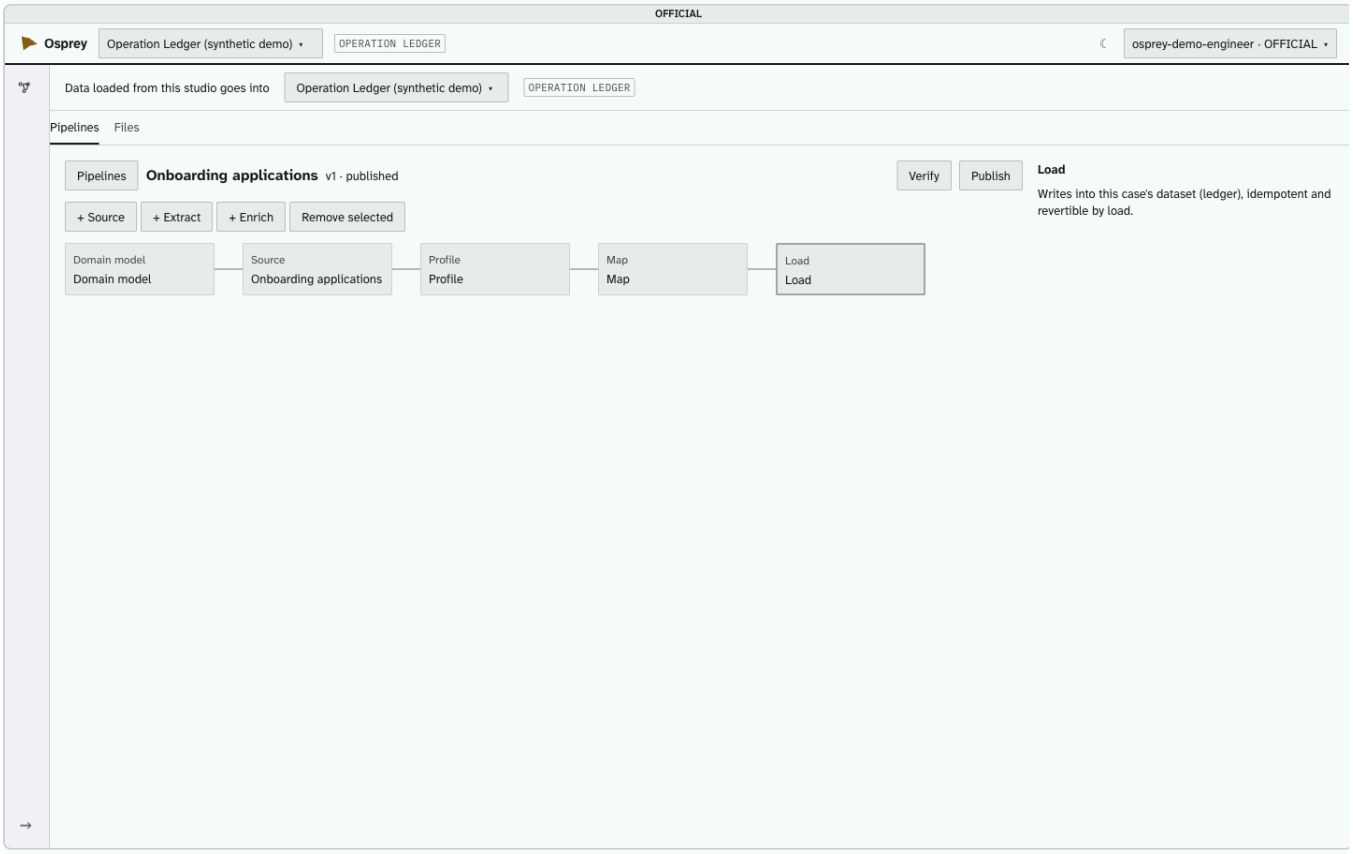


Figure 8. The Load stage

The load writes into the case's dataset. It is **idempotent** — running the same data again adds nothing new — and every element it writes carries its load's id, so a load can be reverted as a whole.

3.8 Verify

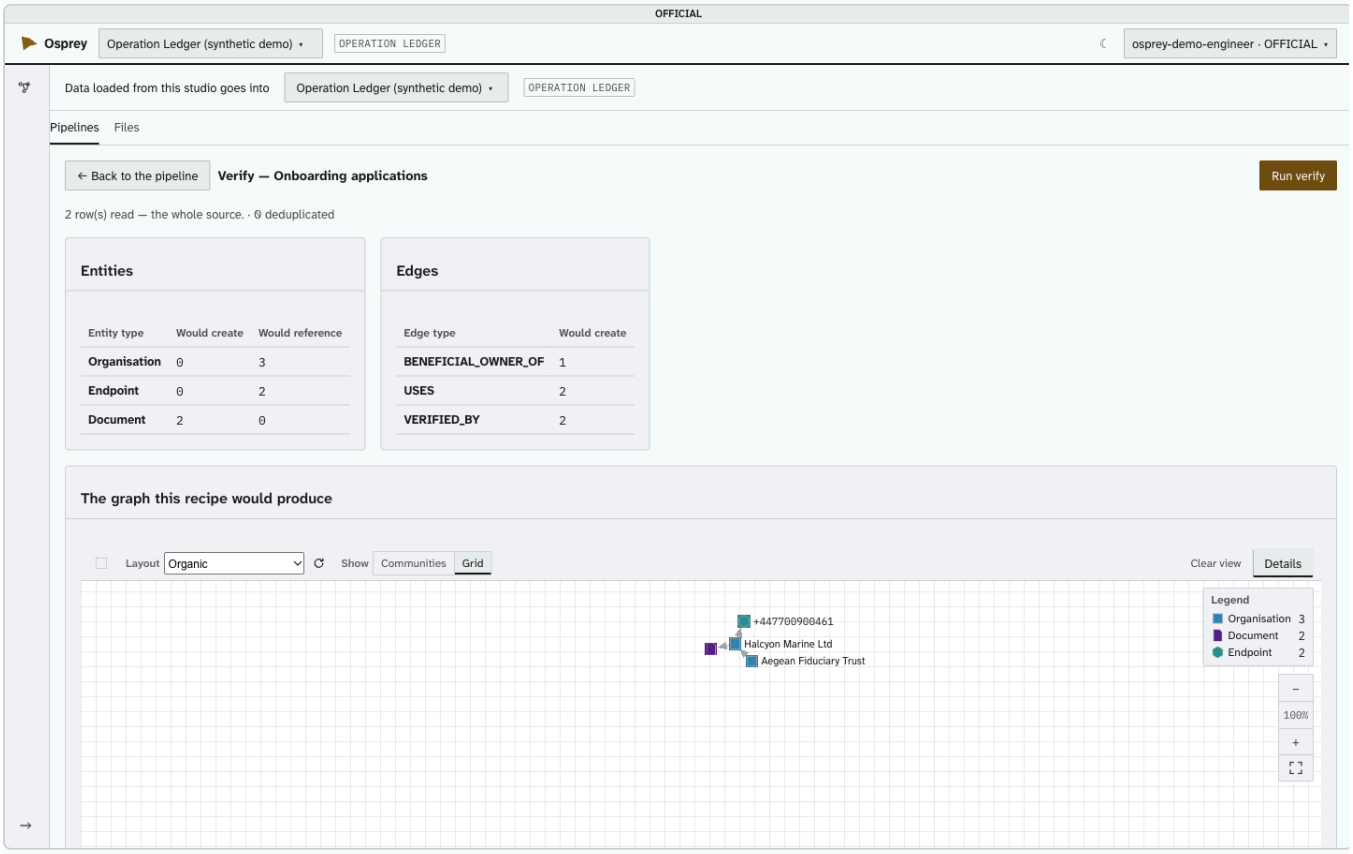


Figure 9. Verify: what the recipe would create and reference, and the graph it would make

Verify is a dry run: nothing is written. It shows, for every type, how many entities and links the recipe **would create**, how many it **would reference** (existing ones, looked up within the case's datasets as you), how many rows were deduplicated or rejected, and the graph itself.

Rejections are grouped by cause, naming the adapter and field — never the value. A value the ontology refuses (a date of birth that is not a date) rejects its row; nothing is silently changed. **A recipe is published only after it has verified.**

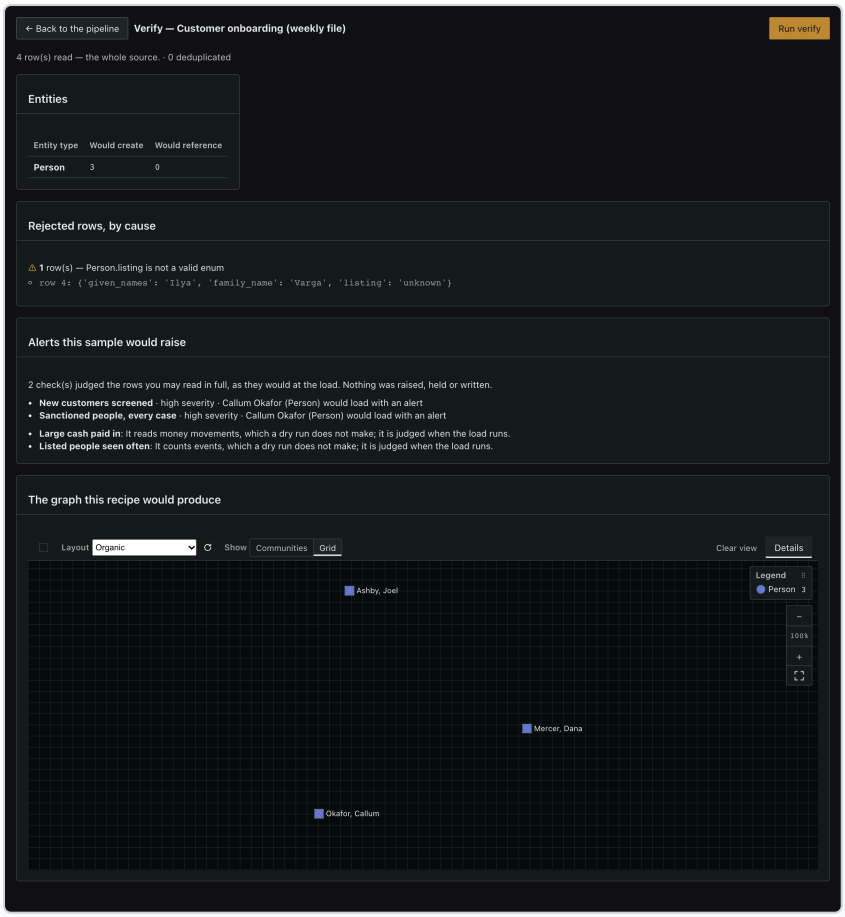


Figure 10. Verify: the alerts the sample would raise under the pipeline's checks, beside the rejections

Alerts this sample would raise. Beside the rejections, Verify judges the sample with the pipeline's checks on load (section 3.8a), each rule as its author, as the load would — and lists, for each alert, the rule, its severity, the entity it names and what a hit would do here: load with an alert, or hold the rows. Nothing is raised, held or written: the run that does this has no way to write. It judges only the rows you may read in full, and names only entities the sample shows you. Some checks cannot be judged in a dry run, and say why: one that reads money movements or counts events (a dry run makes neither), and one that reads links already in the graph. Those are judged when the load runs.

3.8a Checks on load

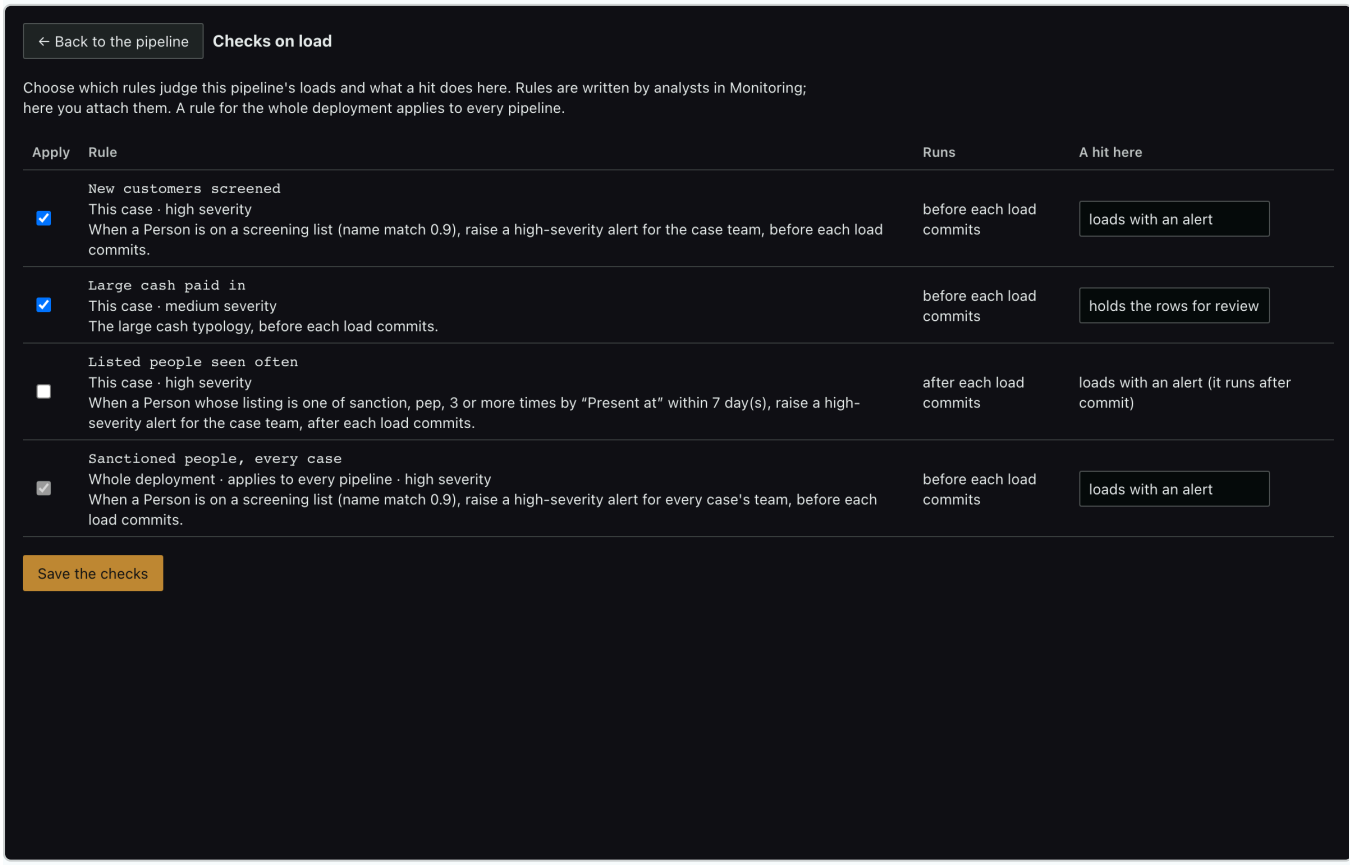


Figure 11. Checks on load: the rules that judge this pipeline's loads, and what a hit does here

Checks on load, a button at the top of a pipeline, chooses which rules judge this source's loads and what a hit does here. The list holds the case's rules and the whole deployment's that judge data as it loads, each with its sentence or typology in words, its severity and when it runs.

- **Apply** attaches a rule to this pipeline. Until anyone chooses, every rule of the case judges the pipeline's loads, as it always has; once you save, only the rules you attached do.
- **A hit here** chooses what happens: **loads with an alert**, or **holds the rows for review** (they wait under Held rows for a reviewer). Holding is offered only for a rule that runs before the load commits.
- **A rule for the whole deployment applies to every pipeline**: it cannot be left off, though you may make its hits hold here. A rule an analyst keeps for themselves is their own watch, not a check on a load, and is not listed.
- A rule waiting for a second person's approval is listed, and runs once approved.

Save the checks records your choice, with your name; it is read at every run, scheduled or not, and when held rows are released. You attach rules here; you do not write them. Rules are written by analysts in Monitoring, as sentences or from templates.

3.9 Publish, versions and running

Publish commits the recipe to `recipes/<id>.yaml` in the deployment's recipe repository and marks it published. To change a published recipe, **Reopen for editing** makes a new draft version; a published version is never edited in place. If the ontology changes under a published recipe, the recipe is **flagged**, and its scheduled runs stop until it is looked at again.

A publish is refused if a stage cannot run here: an extract library this deployment does not have, an unknown geocoder, or a handling override with no reason.

Run now loads the source. Every row is staged in the object store first, then the graph is written, then events go to the analytics database and places to the map. The run's report gives rows read, rows deduplicated, entities created and referenced, links created, rejections, rows held, and what the case's rules said. A run is refused if the source is marked above your clearance.

Known limits: a re-run adds its events to the analytics database a second time (the graph is unaffected); and loads write one element at a time, so a large source takes a while.

4. Files

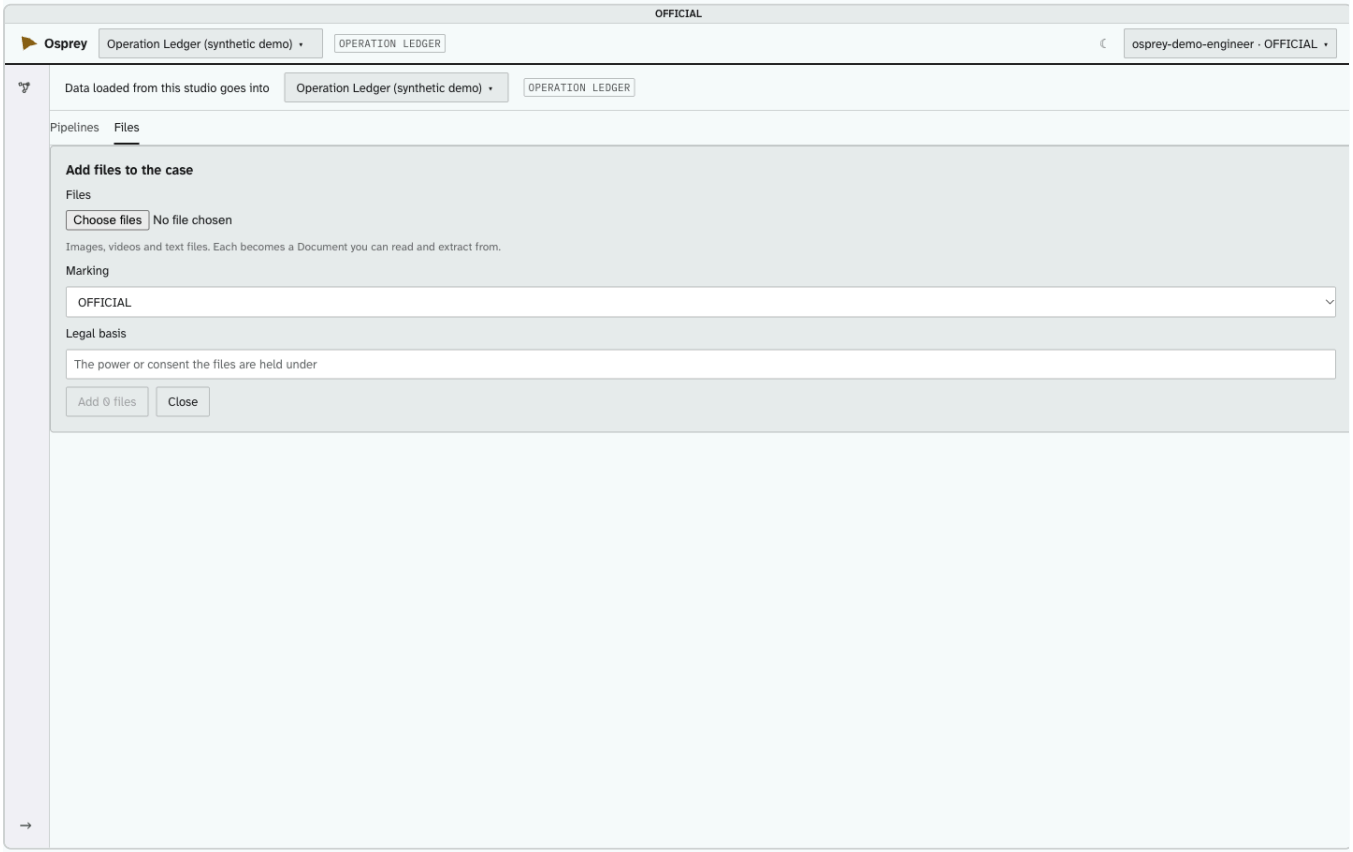


Figure 12. The Studio's Files tab: images, videos and text files added to the case

The Studio's **Files** tab adds images, videos and text files to the case, each becoming a Document that can be read and extracted from (PNG, JPEG, GIF, WebP, TIFF, HEIC, MP4, MOV, WebM and plain text). It takes a marking and a legal basis for the files. PDF and Word documents are not read yet. **Adding files there needs** `extract_files`, which the Data Engineer role does not hold — an analyst adds them, or an administrator grants the right.

5. The ingest folder

The source files a pipeline reads live in the deployment's **ingest folder** (`components.storage.ingest`). OspreyFlow only ever **reads** it, except for its `uploaded/` folder, where files uploaded through the Studio land, under the case's dataset. The Studio lists files directly in the folder, in `uploaded/`, and in each of the case's dataset folders — never anything outside the folder, in deeper folders, or hidden.

An upload is refused, in words, if its name has a folder in it, starts with a dot or is over 200 characters; if it is empty, or over the size limit (512 MiB by default); if its kind is not one OspreyFlow takes; or if a file of that name is already there — **nothing is ever overwritten**.

6. After the load

6.1 Held rows

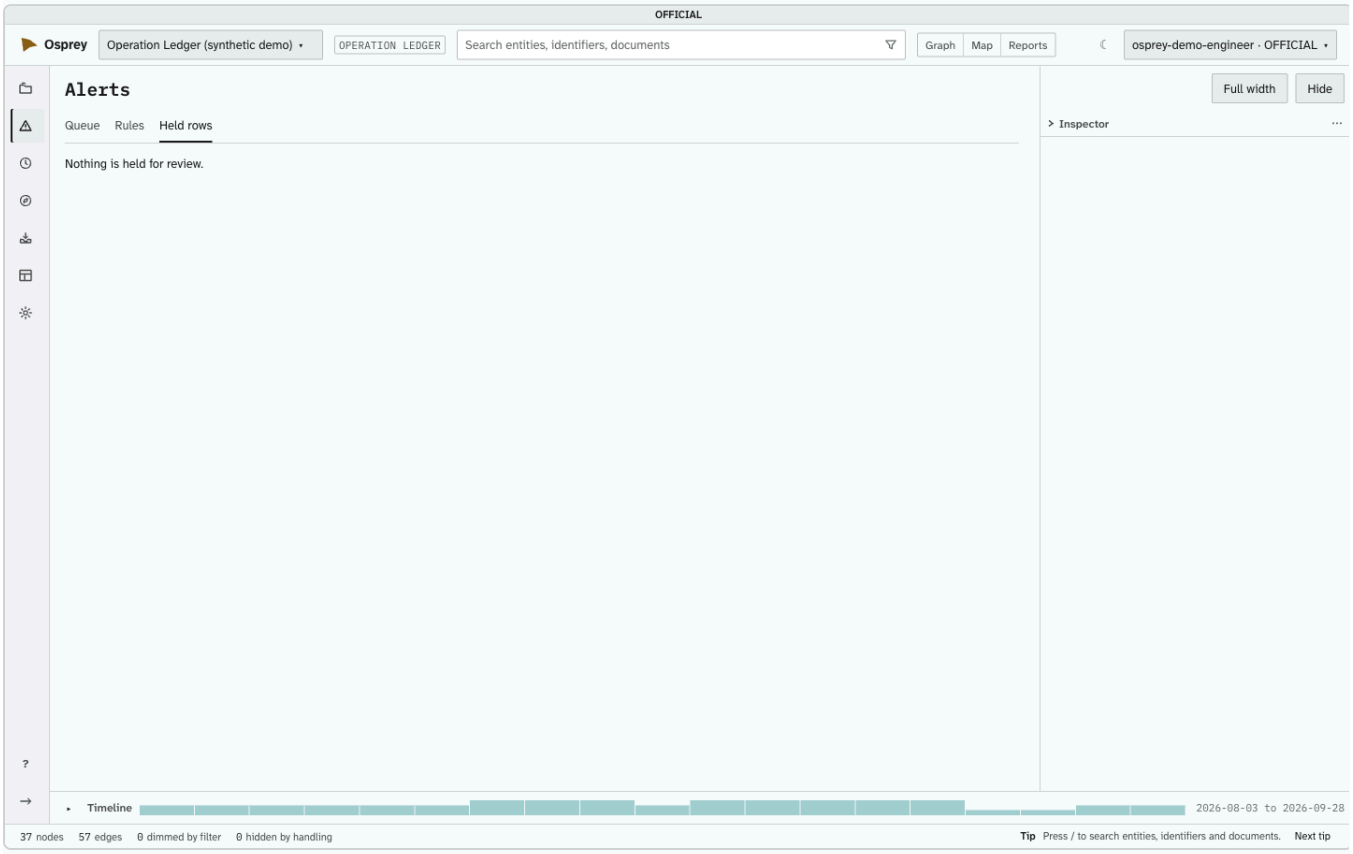


Figure 13. Alerts → Held rows

A case's rules judge every load before it commits. A row a rule could not judge is **held**: it reaches no store until someone decides it, in the Glass under **Alerts → Held rows**, which shows why each row was held. **Release and load** writes it under a new load (it is judged again, and never held a second time); **Reject** needs a written reason. Listing held rows needs Editor on the case; **deciding them needs Author**.

Row numbers. The Glass counts rows from 1: the row under the header is row 1, in held rows, Verify's rejected rows and a source's `row N`. The API and what is stored count from 0: a `row N` locator, a rejection's or held row's `row_index`, and an alert's `fired_on` `row`. A tool of your own that shows one of these to a person adds one. Sample rows (`…/map/sample-rows/{row_index}`) are indexed from 0 too.

6.2 Data feeds

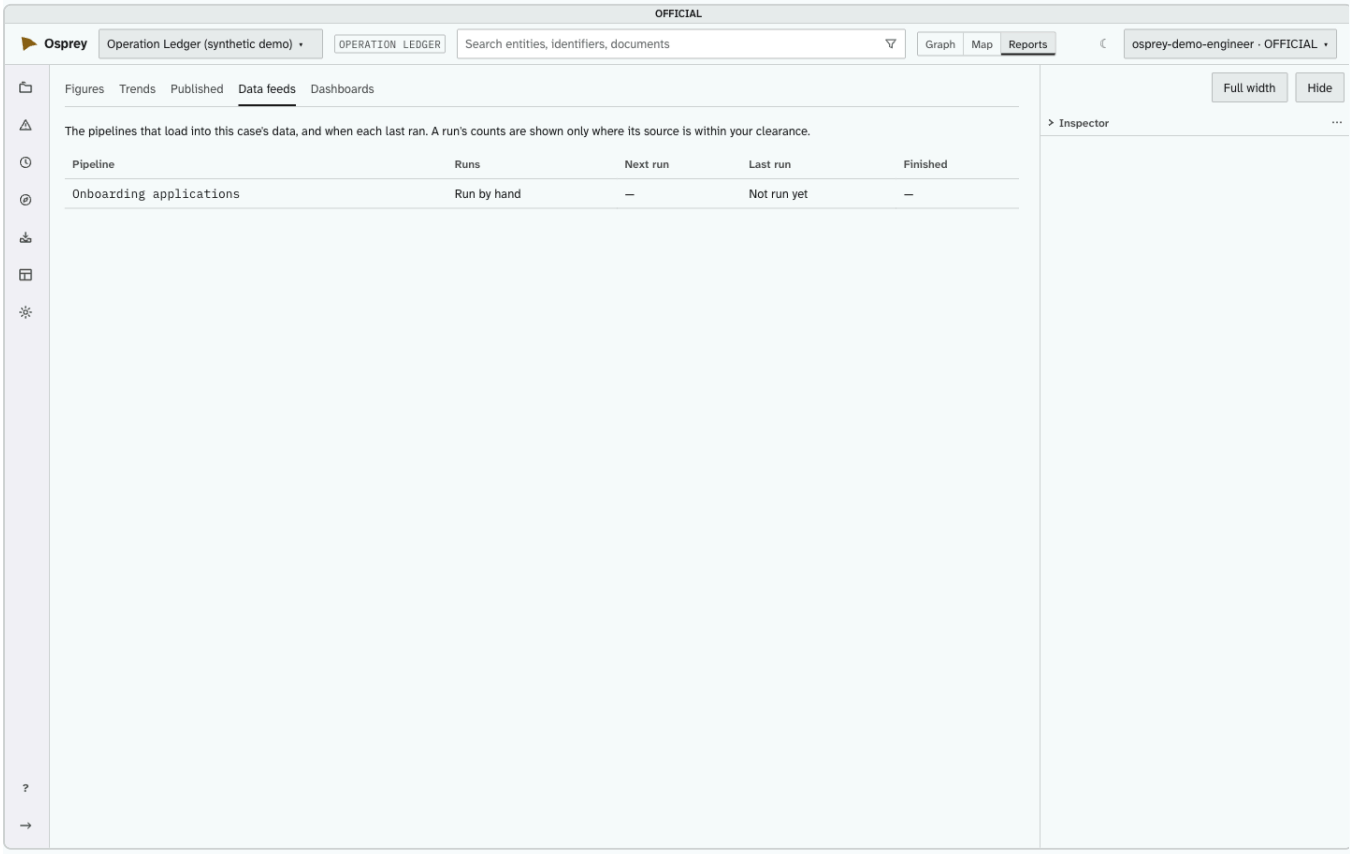


Figure 14. Reports → Data feeds: the pipelines that load into this case, and when each last ran

Under **Reports → Data feeds**, anyone holding `read_data_feeds` with Viewer on a case sees the pipelines that load into it, how they run (by hand or on a schedule), and when each last ran. A run's counts are shown only to someone cleared for its source.

6.3 Schedules and watched folders

A published pipeline can run on a **schedule** — every hour, every day, every weekday, every week, or a cron expression — in a named time zone (Europe/London by default), never more often than every 15 minutes, since each run is a whole load. The screen previews the schedule in words with its next five runs.

It can also run when a **file arrives** in its dataset's own folder, matching a pattern such as `*.csv`: each file is taken once, a minute after it stops changing, one at a time.

Either way it runs **as the person who last set it**, with every check their own run would get. A run that is refused — they can no longer sign in, have lost the right, are no longer an Editor, or are cleared below the source — tells them on their alerts; **three refusals in a row pause it**. Admin → Schedules and runs lists every one.

6.4 Reverting a load

Every element a load writes carries its load's id, and reverting a load hides all of them from every read without deleting anything. **The revert exists in the product's code but is not yet reachable from a screen or a command**; when it is, it will need `revert_loads`.

7. The domain model

7.1 Types

The ontology (`packages/ontology/default.yaml`) defines the entity types — **Person, Endpoint, Device, Location, Organisation, Event, Document, Vehicle, Asset** — and the link types: communicated with, uses, present at, mentioned in, verified by, holds, depicts, associate of, family of, member of, transacted, paired with. A deployment may add types; it cannot add POLE classes. An Endpoint's value is always held normalised, with its kind beside it.

The graph holds **one summarised link per pair per type**, with its first and last time, how often it occurred and its weight; the individual events are in the analytics database.

7.2 What every record carries

Part	What it holds
Provenance	Asserted, inferred or unresolved; its sources (withheld from a reader whose case does not read all of the element's datasets, section 7.5); what extracted it and how sure; when first seen, last confirmed, and by whom
Handling	Classification, handling code, action code, sanitisation code, originator, owning unit, review date, legal basis
Datasets	Which datasets it belongs to; a later load adds, never removes
Audit	The load, recipe and recipe version that wrote it; its revision; whether it was reverted

Handling never goes down. Writing an element again never lowers its marking. A clearance the deployment does not recognise sees nothing.

7.3 Inferred and confirmed

Anything a model produces is **inferred** until a person confirms it; confirming records who and when. Nothing is promoted automatically, and no confidence score does it on anyone's behalf.

7.4 Resolution: never merged

Loads write each source's own records, and **nothing is ever merged in storage**. Resolution links records it thinks are one with a scored `SAME_AS`, and the Conduit shows each linked group as one at the moment it is read, at the case's chosen strictness: Strict (0.99), Balanced (0.95) or Broad (0.80). An analyst can link two records by hand, or keep two apart — which is always honoured. A recipe may not write either.

7.5 Sources withheld for a scoped reader

An element can belong to more than one dataset, and a case reads only the datasets it names. A reader may see an element through any one of its datasets, but its **sources** (the files and rows it came from) name no dataset. So the sources are sent to a reader only when every dataset the element belongs to is one the reader's case reads. Otherwise the reader gets the node or link with its sources emptied and a flag set: what it came from is held back, not the fact that it exists.

Where it is applied. In the Vault, where an element is read for a principal, so that nothing stored or forwarded carries the sources past it:

- The graph adapter's element mapping, `node_from_element_map` and `edge_from_element_map` in `packages/adapters/graph_arcadedb/serialization.py`, which every read path goes through, given the reading principal.
- The working set's projection, `_project` and `project_delta` in `services/api/working_set/service.py`, which serve the replay, the live stream and a restore.
- A principal whose `datasets` is `None` is the Conduit's own unlimited work, and reads every dataset (`services/api/dataset_scope.py`). Nothing a request handler is given is that principal.

The shape. `Provenance.sources_withheld` (`packages/schemas/domain.py`) is `False` by default. When it is `True`, `sources` is empty; when it is `False`, `sources` is never empty. The generated TypeScript types follow. The predicate is one function, `sources_within_case`, in `packages/ontology/handling.py`: every dataset of the element is in the principal's `datasets`; an element that names no dataset, read by a scoped principal, is withheld. `withhold_sources` applies it and returns a copy; it changes nothing stored.

The rule for a writer that reads and writes back. A writer that reads an element and writes its provenance back (a link change, a picture, an extraction's replacement, a resolution, a pipeline run's mention, confirming an alert's cited links) would rebuild the element from a withheld, empty list. That would silently drop the stored sources, or be refused by the write guard below. So **a read whose result is written back runs inside `ontology.handling.sources_in_full()`**:

```
from ontology.handling import sources_in_full

with sources_in_full():
    found = await graph.get_edges([edge_id], principal)
# change what you mean to change in found[0], then write it back
```

The read is still made with the request's principal, so the access filter is applied in the adapter as before; only the sources are left in. The scope is a context variable, so it holds for the calling task alone, and the adapter and its test double honour it. The writers that do this today are `services/api/link_changes.py`, `services/api/entity_pictures.py`, `services/extract/writer.py`, `services/resolve/trigger.py`, `services/api/pipeline_run.py` and `services/api/alert_queue.py`. The working set's own writes are covered the same way. Never wrap a read whose result goes to the Glass.

What such a writer sends back. Whatever a writer returns or streams to the Glass (a delta's `to_sent()`, a route's response) passes through `withhold_sources` for the reader first. A writer that read in full has the sources in its hands, and a response built from them would send the sources the reader may not have. `services/api/link_changes.py` does this for the edge it returns, and `services/api/working_set/models.py` for the deltas it sends.

A withheld provenance is never written. The write guard, `_refuse_withheld` in `packages/adapters/graph_arcadedb/serialization.py`, raises if a `NodeWrite` or `EdgeWrite` carries a `Provenance` with `sources_withheld` set, since writing it would replace the stored sources with none. A refusal there means a read that was written back was not inside `sources_in_full()`.

For a new writer. Ask two questions. Does it read an element and write its provenance back? Then wrap the read. Does it send the element, or anything built from it, to a reader? Then pass it through `withhold_sources` first. The build's tests hold every writer to this: each reads as a scoped principal, changes an element in two datasets, and checks that every stored source is kept and that what is sent back carries none.

7.6 Connection strength: weights, influencers and rarity

A link's **connection strength** is its stored weight with the influencers that apply to it, worked out when it is read. **Shared-attribute rarity** is a separate measure beside it. Both are computed in the graph adapter, as the reader, and neither is ever stored (ADR-139; spec 01 §6.1a).

The stored weight (weights version 3, `packages/ontology/weights/default.yaml`):

```
weight = normalise( α·log1p(occurrences)/log1p(heavy(type)) + β·recency(last seen) )
normalise(x) = 1 / (1 + e^(−(x − midpoint)/scale)),   recency(t) = 0.5^(age/90 days)
```

with α = 0.5, β = 0.2, `scale` = α/3 and `midpoint` = β + α/3, or a structural type's fixed weight. It comes from the link alone (ADR-123): no other link's count reaches it.

Influencers (`packages/schemas/strength.py`, `StrengthInfluencer`):

Field	Meaning
`id`, `label`	Its key (lower case, `-` and `_`) and what the legend says
`trigger`	`{entity_type, property, values}`: a record of that type whose property holds one of the values, at one end of the link or linked to both ends. Omitted: a **direct-links** influencer, applying to its link types always
`link_types`	The link types it changes
`change`, `size`	`add` (a boost to the pre-logistic score, above 0 and at most 2), `multiply` (0.1 to 5), `floor` (a strength, above 0 and at most 1) or `exclude` (size 0: no stronger than the stored weight)
`enabled`	`false` in a later layer switches off the influencer of the same id beneath it
`note`	Shown beside it, such as "to be confirmed with the client"

Applied to a link of stored weight `w` (`ontology/influencers.py`, `connection_strength`):

```
raw      = midpoint + scale · ln(w / (1 − w))       # the logistic's inverse
strength = normalise( (raw + Σ add) · Π multiply )
strength = max(strength, every floor)
strength = min(strength, w)   if any exclude
```

With nothing applying, the strength is `w`. Each step rises with `w`, so adding occurrences never lowers a strength, whatever influencers apply (`tests/ontology/test_influencers.py` proves it over a grid of every combination).

Resolution order: a case override, then the installation set, then the weights file. Influencers are matched by `id`; a later layer's replaces an earlier one's, and a case may switch any off by id. The installation set is maker–checker (the right `manage_strength_influencers`; a second holder approves, never the proposer; every version kept in `strength_influencer_versions`). A case override is its Author's (`case_strength_influencers`, every version kept). Both are audited.

Read time, under access. `GraphPort.connection_strengths(edge_ids, principal, influencers=…)` reads the links as `get_edges` does (the link and both ends visible), then, for each triggered influencer, finds the evidencing records with the access step on every vertex and every link it steps over. A record the reader may not see, or one reached only over a link they may not see, is not found, so it changes nothing: the strength is the strength in a world without it. The conformance suite's

`test_unauthorised_connection_strength_matches_a_world_without_the_hidden_evidence` holds every adapter to it.

The rarity query. `GraphPort.shared_attribute_rarity(node_ids, principal, scope=…)`, for two visible entities of one type:

```
rarity(u, v) = Σ over shared attributes a of ln( N_visible / df_visible(a) )
```

An attribute is a visible node (of no type in the file's `rarity.exclude_types`) linked to each of the two by a visible link; `N_visible` counts the visible entities of the pair's type in scope; `df_visible(a)` counts those linked to `a` by a visible link. **Per dataset** (the default) it is worked out in each dataset the two and the attribute all belong to, the highest kept and named; **across the case**, over every dataset the reader may read. In ArcadeDB it is three reads: the entities and their attribute ids, each attribute's visible neighbours with their types and datasets, and the visible count per type and dataset. More pairs to score than the cap (2,000,000) is refused as `limit_exceeded`, never scored in part. The Glass is sent the score and the visible shared attributes, never `N` or `df`.

Shipping influencers in the weights file. The developer defaults sit beside the weight versions, under `influencers:`; they are not a weight version and change no stored weight:

```
influencers:
  - id: direct
    label: Direct link
    link_types: [COMMUNICATED_WITH, TRANSACTED, VISITED, SHARED_CELL_WITH]
    change: add
    size: 0.1348
    note: to be confirmed with the client
rarity:
  exclude_types: [Alert]
```

A shipped boost's size is calibrated, not tuned: how far doubling an ordinary link's count (from the default rule's heavy count, 10, to 20) moves its pre-logistic score, α·(ln 21 – ln 11)/ln 11 = 0.1348. `test_influencers.py` holds the shipped sizes to it, and the loader refuses a malformed influencer by its position and id.

8. Screening lists

Sanctions, PEP and adverse-media lists are kept beside the graph, not in it, deployment-wide, and replaced a whole list at a time. They are loaded from a YAML file on the server:

```
python -m scripts.load_screening_list <file.yaml>
```

The file names the list, whether it is synthetic, and its entries — each with an id, a listing (`sanction`, `pep` or `adverse_media`), an entity type (Person or Organisation), a name, and optionally a date of birth and country. **A list of real people is refused** unless the deployment's local real-data exceptions name its dataset. A match reaches the graph only as a scored, inferred `SAME_AS`, for a person to confirm.

9. Interfaces

9.1 The API

Every route is under `/v1`; a case's routes are under `/v1/investigations/{investigation_id}`. A missing right is refused with `403` and the right's name; a case you have no level on answers as if it did not exist.

The OpenAPI document describes every route: `GET /v1/openapi.json` at the deployment's own address, signed in like every other route. Its `info.version` is the release the deployment runs (for example `v0.4.1`, or `dev-<commit>` off a release), the same version Admin → System shows.

Method and path	Needs
`GET /v1/ingest/connectors`, `/v1/ingest/packs`, `/v1/ingest/transforms`	Signed in
`GET`, `POST …/ingest/source-files`	`add_ingest_files` (upload: Editor)
`POST …/ingest/recipes`; `PATCH`, `GET …/ingest/recipes/{recipe_id}`; `GET …/ingest/recipes`	`edit_pipelines`
`PUT …/ingest/recipes/{recipe_id}/pipeline`	`edit_pipelines`, Editor
`GET …/ingest/recipes/{recipe_id}/profile`	`edit_pipelines`
`PUT …/row-template`; `GET …/map/sample-rows/{row_index}`; `POST …/map/handling-preview`	`edit_pipelines`
`POST …/ingest/recipes/{recipe_id}/verify`	`verify_pipelines`, Editor
`POST …/ingest/recipes/{recipe_id}/publish`; `POST …/versions`	`publish_pipelines` / `edit_pipelines`, Editor
`POST …/ingest/recipes/{recipe_id}/run`	`run_pipelines`, Editor
`PATCH …/schedule`; `POST …/schedule/pause`, `…/resume`; `POST /v1/ingest/schedule-preview`	`schedule_pipelines`
`GET …/ingest/recipes/{recipe_id}/runs`; `GET …/data-feeds`	`read_data_feeds`, Viewer
`GET …/held-rows`	`read_held_rows`, Editor
`POST …/ingest/recipes/{recipe_id}/held-rows/release`; `POST …/held-rows/reject`	`decide_held_rows`, Author
`GET /v1/admin/schedules` and its actions	`operate_schedules`
`POST …/connection-strength`; `POST …/rarity`	`investigate`, Viewer: strengths and rarity over the working set, as the reader
`GET …/strength-influencers`	`investigate`, Viewer: the legend and the case's override
`PUT …/strength-influencers`	`investigate`, Author: the case's override
`GET`, `POST /v1/strength-influencers`; `POST /v1/strength-influencers/{version}/approve`, `…/send-back`	`manage_strength_influencers`; never the proposer's approval
`GET /v1/me/rights`	Signed in: your own rights
`GET /v1/openapi.json`	Signed in: the OpenAPI document

9.2 Files on the server

Where	What
The ingest folder (`components.storage.ingest`)	Source files, read-only to OspreyFlow; `uploaded/` beneath it is where uploads land
`recipes/<id>.yaml`	Each published recipe
`connectors/<id>.yaml`	Each available connector's profile
`packages/ontology/default.yaml`, `packs/`	The ontology and its domain packs

9.3 Commands

Command	What it does
`ospreyflow demo list`, `ospreyflow demo add <id>`	The synthetic demo packs, each loaded into its own case
`python -m scripts.seed_ingest_folder`	Writes the demo packs' synthetic source files into the ingest folder
`python -m scripts.load_screening_list <file.yaml>`	Loads or replaces a screening list (section 8)